

Chapter 5: Threads Basics

Processes are management objects, and do not execute code directly. To get anything done on Windows, threads must be created. As we've seen already, a user mode process is created with a single thread that eventually executes the main entry point of the executable. In many cases, this is good enough, and the application may not require any more threads.

Some applications, however, may benefit from using multiple threads executing within the process. Each thread is an independent path of execution, and so can use a different processor, resulting in true concurrency. In this chapter, we'll explore the fundamentals of creating and managing threads. In subsequent chapters, we'll delve into other aspects of threads, such as scheduling and synchronization.

In this chapter:

- **Introduction**
 - **Creating and Managing Threads**
 - **Terminating Threads**
 - **A Thread's Stack**
 - **A Thread's Name**
 - **What About the C++ Standard Library**
-

Introduction

The first question we should consider is why use threads in the first place? There are essentially two possible reasons:

1. Increase performance by utilizing multiple cores executing concurrently.
2. Improve application design.

Although you may come up with other reasons to use threads, these reasons somehow fall into the second category. It's always possible to design with a single thread (maybe by using timers, for example), without creating more threads. Still, the second reason is valid and is in fact the dominant one. A quick look at *Task Manager* in the *Performance/CPU* tab shows many (thousands of) threads, much more than the number of processors, and still the consumed CPU percentage at any time is low, meaning reason number 1 is not dominant (figure 5-1).

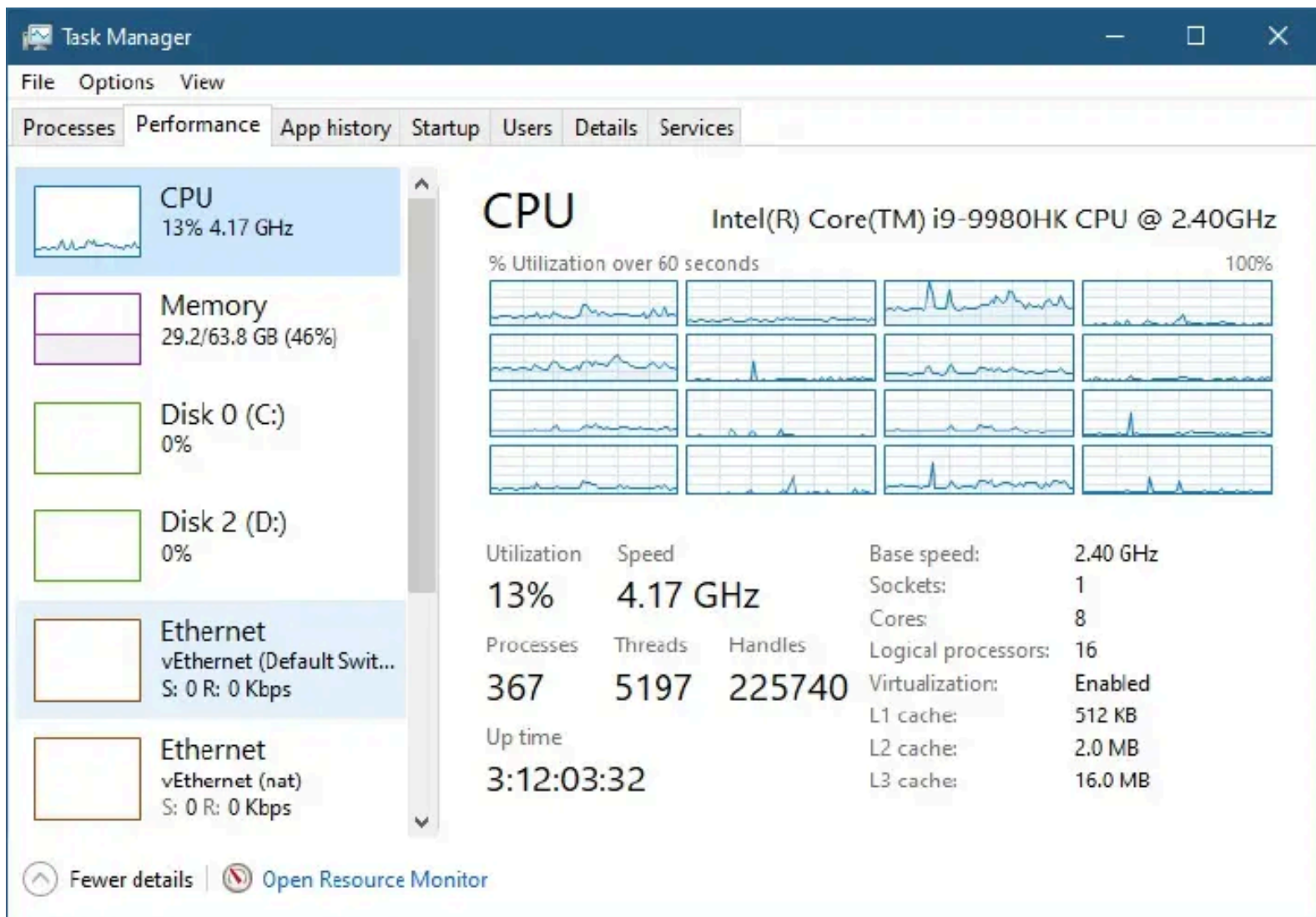


Figure 5-1: Performance/CPU in *Task Manager*

A thread abstracts an independent execution path, unrelated (from an execution perspective) to other threads that may be active at the same time. Once a thread starts execution, it may perform any of the following operations, until it exits:

- CPU-bound operations - calculations or invocation of functions that rely on CPU operations to make progress.
- I/O bound operations - operations performed against I/O devices, such as disks or the network. While waiting for an I/O operation to complete, the thread is in a wait state,

and does not consume CPU cycles.

- Other operations that may result in the thread entering the wait state, such as waiting on a synchronization primitive (e.g. a mutex).

Thread Synchronization is discussed in detail in chapter 7.

The fact that the CPU utilization in figure 5-1 is not 100% means that most threads are in a wait state (do not want to execute). In fact, if 16 threads execute code at the same time on the machine (figure 5-1), the CPU utilization would be 100%. It's only about 13%, which means there are about 2 processors active at the same time.

Sockets, Cores and Logical Processors

Before we get any further with threads, we must realize that threads are an abstraction over processors. But what exactly is the definition of a processor? In the days where multiple cores make up a typical CPU, the terms might become confusing. Figure 5-2 shows a logical composition of a typical CPU.

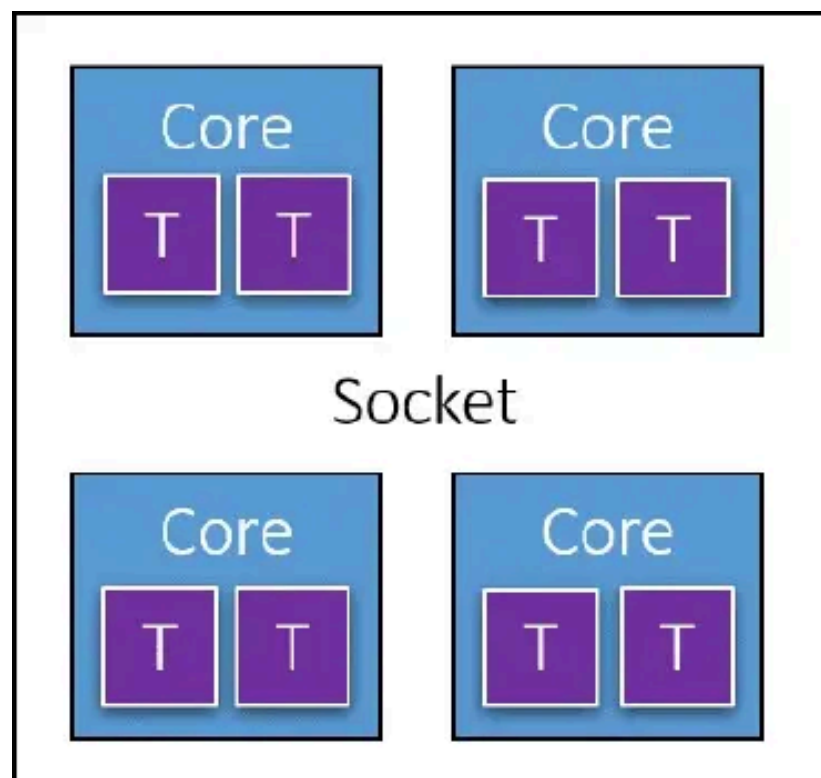


Figure 5-2: Logical composition of a CPU

In figure 5-2, there is one *socket*, which is the physical chip stuck in the computer's motherboard. Laptops and home computers typically have just one of these. Large server machines may contain more than one socket. Each socket has multiple *cores*, which are independent processors (4 in figure 5-2).

On Intel processors, each core may be split to two *logical processors*, also called *hardware threads* because of a technology called *Hyper-threading*. From Windows perspective, the number of processors is the number of logical processors (16 in figure 5-1). This means that at any given moment, at most 16 threads may be running. The number of sockets, cores and logical processors is also shown by *Task Manager* (figure 5-1).



AMD has a similar technology called *Simultaneous Multi Threading* (SMT).

Hyper-threading can be disabled in the BIOS settings. The potential downside of hyper-threading is that every two logical processors that share a core also share the Level 2 cache, and so might “interfere” with each other. Chapter 6 has more to say of caching.

Creating and Managing Threads

The basic function to create a thread is `CreateThread`:

```
HANDLE WINAPI CreateThread(
    _In_opt_ LPSECURITY_ATTRIBUTES lpThreadAttributes,
    _In_     SIZE_T dwStackSize,
    _In_     LPTHREAD_START_ROUTINE lpStartAddress,
    _In_opt_ LPVOID lpParameter,
    _In_     DWORD dwCreationFlags,
    _Out_opt_ LPDWORD lpThreadId);
```

The first argument to `CreateThread` should be recognizable by now, typically set to `NULL`. The `dwStackSize` parameter sets the thread's stack size, discussed in detail in the section “A Thread's Stack”, later in this chapter. It's usually set to zero, which sets a default sizes

based on the PE header. I say “sizes” because the stack has an initial size and a maximum size (discussed later as well).

The `lpStartAddress` parameter is the most important one, specifying the user function to call from the new thread. This function can be named anything but it must adhere to the following prototype:

```
DWORD WINAPI ThreadProc(_In_ PVOID pParameter);
```

The thread function must return a 32-bit number, which is considered the thread’s exit code, that can later be retrieved with `GetExitCodeThread`. The `WINAPI` macro expands to the `__stdcall` keyword, signifying the standard calling convention which is common to most Windows APIs. Finally, the parameter passed to the function is a user-defined value that is passed as the fourth parameter to `CreateThread`, and simply passed as-is to the thread function. This value typically points to some data structure containing information that allows the thread to do its job.

Back to `CreateThread` - the `lpParameter` parameter was just discussed. In the simplest cases, `NULL` could be passed in. The `dwCreationFlags` argument can have three possible values (which can be combined). Specifying the `CREATE_SUSPENDED` flag creates the thread in a suspended state. The thread is ready to go, but a call to `ResumeThread` must be made to let it loose. Another possible value is `STACK_SIZE_PARAM_IS_A_RESERVATION`, which gives an alternate meaning to the stack size parameter (discussed in the section “A Thread’s Stack” as well). Finally, specifying neither of these flags (most common), instructs the thread to start execution immediately. The last optional argument to `CreateThread` is the resulting unique thread ID of the new thread. If the caller is not interested in this information, it can simply specify `NULL` for this argument.

The return value from `CreateThread` is a handle to the newly created thread. If something goes wrong, the return value is `NULL` and `GetLastError` can be called to extract the error code. Once the handle is not needed, it should be closed with `CloseHandle` like any other kernel object handle.

The following code snippet creates a thread from the main function, waits for it to exit, and prints out its exit code:

```
DWORD WINAPI DoWork(PVOID) {
    printf("Thread ID running DoWork: %u\n", ::GetCurrentThreadId());
    // simulate some heavy work...
    ::Sleep(3000);
    // return a result
    return 42;
}

int main() {
    HANDLE hThread = ::CreateThread(nullptr, 0, DoWork, nullptr, 0, nullptr);
    if(!hThread) {
        printf("Failed to create thread (error=%d)\n", ::GetLastError());
        return 1;
    }

    // print ID of main thread
    printf("Main thread ID: %u\n", ::GetCurrentThreadId());

    // wait for the thread to finish
    ::WaitForSingleObject(hThread, INFINITE);

    DWORD result;
    ::GetExitCodeThread(hThread, &result);
    printf("Thread done. Result: %u\n", result);

    ::CloseHandle(hThread);
    return 0;
}
```

Here is an example output:

```
Main thread ID: 19108
Thread ID running DoWork: 23700
Thread done. Result: 42
```

The `GetExitCodeThread` allows retrieving the returned value from a thread's function:

```
BOOL GetExitCodeThread(  
    _In_ HANDLE hThread,  
    _Out_ LPDWORD lpExitCode);
```

You might be wondering what would happen if the function is called with a thread that has not exited yet. The function does not fail, but returns `STILL_ACTIVE` (0x103=259).

The Primes Counter Application

The following example illustrates a more complex usage of multiple threads. The *PrimesCounter* application (available in the samples for this chapter), counts the number of prime numbers in a range of numbers using a specified number of threads. The idea is to split the work between several threads, each counting prime numbers in its range of numbers. Then the main thread waits for all worker threads to exit, allowing it to simply sum the counts from all threads. This is depicted in figure 5-3.

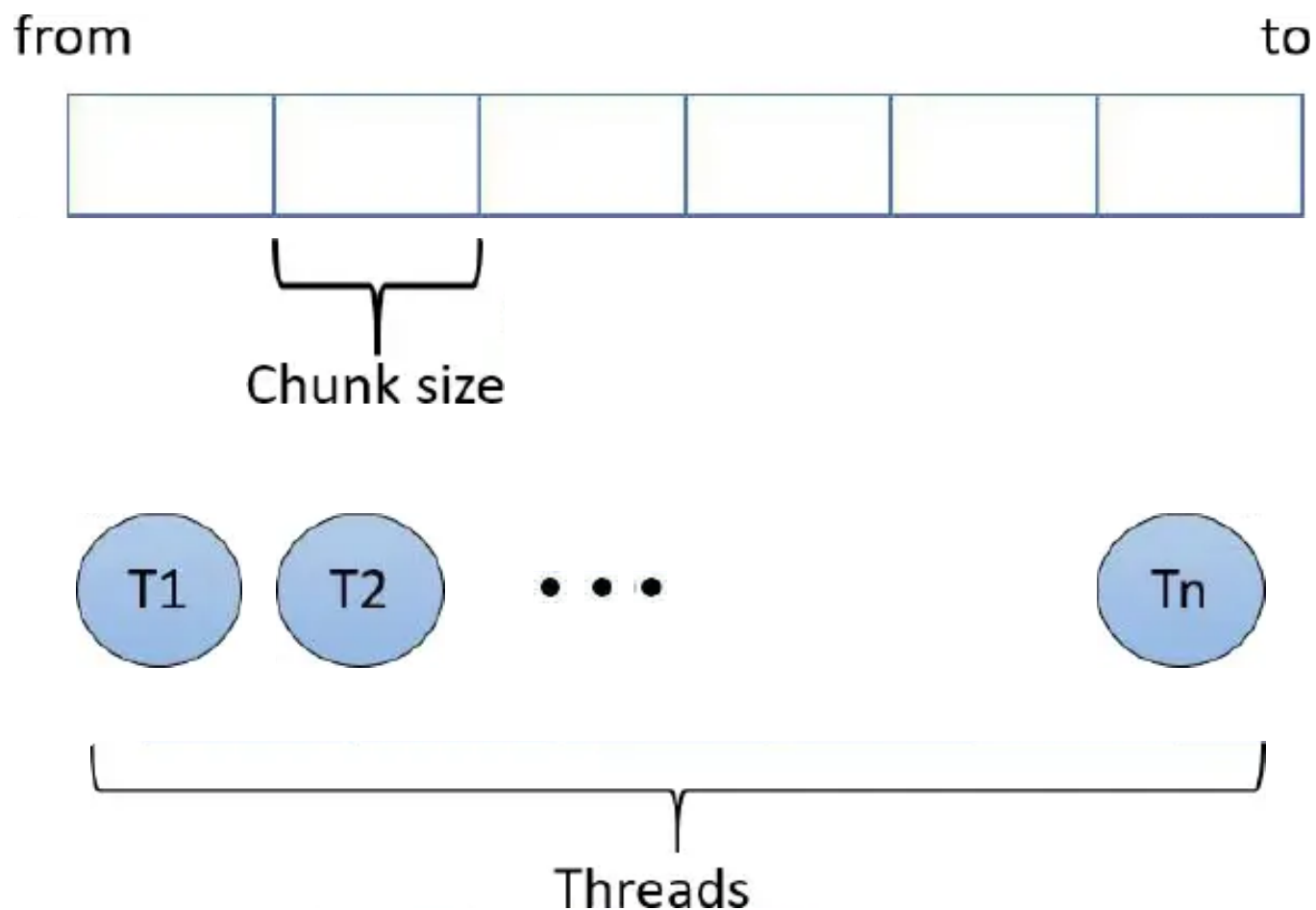


Figure 5-3: *Primes Counter* design

This idea of creating several threads that do some work, and waiting for them to exit before aggregating the results is sometimes called *Fork-Join*, because threads are “forked” from some initial thread and then “joined back” to the initial thread upon completion.

Another name for this pattern is *Structured Parallelism*.

The number of threads used in this application is one of the parameters of the algorithm - the interesting question is what number of threads is optimal to complete the calculation the quickest? This will be discussed later; but, first - the code.

The main function accepts the range of numbers and the number of threads on the command line:

```
int main(int argc, const char* argv[]) {
    if (argc < 4) {
        printf("Usage: PrimesCounter <from> <to> <threads>\n");
        return 0;
    }

    int from = atoi(argv[1]);
    int to = atoi(argv[2]);
    int threads = atoi(argv[3]);

    if (from < 1 || to < 1 || threads < 1 || threads > 64) {
        printf("Invalid input.\n");
        return 1;
    }
}
```

The number of threads is limited to 64. Why this number? This is the maximum number of handles that can be waited upon at the same time with `WaitForMultipleObjects`, which is used later to wait until all threads exit.

The next call in main is to the function that initiates the work and returns result:

```
DWORD elapsed;
int count = CalcAllPrimes(from, to, threads, elapsed);
printf("Total primes: %d. Elapsed: %d msec\n", count, elapsed);
```

CalcPrimes accepts the arguments extracted from the command line and returns the total primes counted, and also returns the elapsed time in milliseconds using the last elapsed parameter (passed by reference). Finally, the results are echoed to the console.

Each thread needs its “from” and “to” numbers and somewhere to put the result. Since a thread function can return a 32-bit unsigned integer, this can be used here. But in the general case, the return value may not be flexible enough. The typical solution is to define a structure that has all the information required by the thread, including input and output values. For our application, the following structure is defined:

```
struct PrimesData {
    int From, To;
    int Count;
};
```

CalcAllPrimes must allocate a PrimesData instance for each thread and initialize the From and To data members:

```
int CalcAllPrimes(int from, int to, int threads, DWORD& elapsed) {
    auto start = ::GetTickCount64();
    // allocate data for each thread
    auto data = std::make_unique<PrimesData[]>(threads);
    // allocate an array of handles
    auto handles = std::make_unique<HANDLE[]>(threads);
```

The current time is captured before any work is done with GetTickCount64. This API returns the number of milliseconds elapsed since Windows booted. Although it’s not the most accurate API to use (QueryPerformanceCounter is more accurate), it will do for this application’s purposes.



GetTickCount64 replaces the older GetTickCount by returning a 64 bit number as opposed to a 32-bit number returned from GetTickCount. A 32-bit number of milliseconds will overflow and roll back to zero after about 49.7 days.

The code uses `std::unique_ptr<[]>` to managed an array that is cleaned up automatically when the variable goes out of scope. This is used for the `PrimesData` array as well as for the threads handles.

Next, the function calculates the chunk size for each thread and then loops to create the threads appropriately:

```
int chunk = (to - from + 1) / threads;

for (int i = 0; i < threads; i++) {
    auto& d = data[i];
    d.From = i * chunk;
    d.To = i == threads - 1 ? to : (i + 1) * chunk - 1;

    DWORD tid;
    handles[i] = ::CreateThread(nullptr, 0, CalcPrimes, &d, 0, &tid);
    assert(handles[i]);
    printf("Thread %d created. TID=%u\n", i + 1, tid);
}
```

Each thread's `PrimesData` instance is initialized with the correct `From` and `To` based on the chunk size. The only wrinkle is that the range may not be exactly divisible by the number of threads. So, the last thread is tasked with the “tail” of numbers (if any). `CreateThread` is called to create each thread, pointing each thread to the `CalcPrimes` function (discussed shortly), passing to it its personal `PrimesData` pointer. Finally, the thread index and ID are displayed.

`CalcPrimes` is the thread function tasked with counting the prime numbers in the range supplied to that thread:

```
DWORD WINAPI CalcPrimes(PVOID param) {
    auto data = static_cast<PrimesData*>(param);
    int from = data->From, to = data->To;
    int count = 0;
    for (int i = from; i <= to; i++)
        if (IsPrime(i))
            count++;

    data->Count = count;
}
```

```
    return count;  
}
```

The parameter passed in to a thread is cast to a `PrimesData` pointer. Then a simple for loop checks if the number is a prime one and if so increments a counter that is eventually stored in the `PrimesData`'s `Count` member. `IsPrime` is a simple function returning true for a prime number and false otherwise:

```
bool IsPrime(int n) {  
    if (n < 2)  
        return false;  
    if(n == 2)  
        return true;  
  
    int limit = (int)::sqrt(n);  
    for (int i = 2; i <= limit; i++)  
        if (n % i == 0)  
            return false;  
  
    return true;  
}
```

The algorithm used in `IsPrime` is certainly not optimal, but that's not the point.

Back in `CalcAllPrimes`, all threads are created without the `CREATE_SUSPENDED` flag, so they start immediately. All that remains is to wait until all threads exit:

```
::WaitForMultipleObjects(threads, handles.get(), TRUE, INFINITE);
```

A full discussion of the wait functions is saved for chapter 7. `WaitForMultipleObjects` above accepts the following arguments, in order:

- the number of handles in the array
- the array of handles to wait on

- a boolean flag indicating if the wait should be for all handles to become signaled (TRUE) or just one (FALSE). For threads, the meaning of “signaled” is “exited”.
- a timeout in milliseconds, in this case INFINITE means wait as long as it takes.

Once all threads exit, the wait is over. All that’s left is to gather results:

```
elapsed = static_cast<DWORD>(::GetTickCount64() - start);

FILETIME dummy, kernel, user;
int total = 0;
for (int i = 0; i < threads; i++) {
    ::GetThreadTimes(handles[i], &dummy, &dummy, &kernel, &user);
    int count = data[i].Count;
    printf("Thread %2d Count: %7d. Execution time: %4u msec\n",
        i + 1, count,
        (user.dwLowDateTime + kernel.dwLowDateTime) / 10000);
    total += count;
    ::CloseHandle(handles[i]);
}
return total;
```

The code above uses the `GetThreadTimes` API to retrieve timing information for a thread:

```
BOOL GetThreadTimes(
    _In_ HANDLE hThread,
    _Out_ LPFILETIME lpCreationTime,
    _Out_ LPFILETIME lpExitTime,
    _Out_ LPFILETIME lpKernelTime,
    _Out_ LPFILETIME lpUserTime);
```

The function returns a thread’s creation time, exit time, the time it spent executing in kernel mode and the time it spend executing in user mode. For this application, I wanted to display the execution time, which means summing up kernel time and user time, while disregarding create and exit times.

The kernel and user times are reported in a `FILETIME` structure, which is a 64-bit value stored in two 32-bit values:

```
typedef struct _FILETIME {
    DWORD dwLowDateTime;
    DWORD dwHighDateTime;
} FILETIME, *PFILETIME, *LPFILETIME;
```

The value is in 100 nanosecond units (10 to the -7th power), which means the value in milliseconds can be obtained by dividing by 10000. The code assumes the elapsed time is not higher than a 32-bit value in 100 nanosecond units, which may not be correct in the general case.

Running Primes Counter

Here are a few runs for the same value range, starting from a baseline with one thread:

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 1
Thread 1 created (3 to 20000000). TID=29760
Thread 1 Count: 1270606. Execution time: 9218 msec
Total primes: 1270606. Elapsed: 9218 msec
```

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 2
Thread 1 created (3 to 10000001). TID=22824
Thread 2 created (10000002 to 20000000). TID=41816
Thread 1 Count: 664578. Execution time: 3625 msec
Thread 2 Count: 606028. Execution time: 5968 msec
Total primes: 1270606. Elapsed: 5984 msec
```

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 4
Thread 1 created (3 to 5000001). TID=52384
Thread 2 created (5000002 to 10000000). TID=47756
Thread 3 created (10000001 to 14999999). TID=42296
Thread 4 created (15000000 to 20000000). TID=34972
Thread 1 Count: 348512. Execution time: 1312 msec
Thread 2 Count: 316066. Execution time: 2218 msec
Thread 3 Count: 306125. Execution time: 2734 msec
Thread 4 Count: 299903. Execution time: 3140 msec
Total primes: 1270606. Elapsed: 3141 msec
```

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 8
```

```
Thread 1 created (3 to 2500001). TID=25200
Thread 2 created (2500002 to 5000000). TID=48588
Thread 3 created (5000001 to 7499999). TID=52904
Thread 4 created (7500000 to 9999998). TID=18040
Thread 5 created (9999999 to 12499997). TID=50340
Thread 6 created (12499998 to 14999996). TID=43408
Thread 7 created (14999997 to 17499995). TID=53376
Thread 8 created (17499996 to 20000000). TID=33848
Thread 1 Count: 183071. Execution time: 578 msec
Thread 2 Count: 165441. Execution time: 921 msec
Thread 3 Count: 159748. Execution time: 1171 msec
Thread 4 Count: 156318. Execution time: 1343 msec
Thread 5 Count: 154123. Execution time: 1531 msec
Thread 6 Count: 152002. Execution time: 1531 msec
Thread 7 Count: 150684. Execution time: 1718 msec
Thread 8 Count: 149219. Execution time: 1765 msec
Total primes: 1270606. Elapsed: 1766 msec
```

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 16
```

```
Thread 1 created (3 to 1250001). TID=50844
Thread 2 created (1250002 to 2500000). TID=9792
Thread 3 created (2500001 to 3749999). TID=12600
Thread 4 created (3750000 to 4999998). TID=52804
Thread 5 created (4999999 to 6249997). TID=5408
Thread 6 created (6249998 to 7499996). TID=42488
Thread 7 created (7499997 to 8749995). TID=49336
Thread 8 created (8749996 to 9999994). TID=13384
Thread 9 created (9999995 to 11249993). TID=41508
Thread 10 created (11249994 to 12499992). TID=12900
Thread 11 created (12499993 to 13749991). TID=39512
Thread 12 created (13749992 to 14999990). TID=3084
Thread 13 created (14999991 to 16249989). TID=52760
Thread 14 created (16249990 to 17499988). TID=17496
Thread 15 created (17499989 to 18749987). TID=39956
Thread 16 created (18749988 to 20000000). TID=31672
Thread 1 Count: 96468. Execution time: 281 msec
Thread 2 Count: 86603. Execution time: 484 msec
Thread 3 Count: 83645. Execution time: 562 msec
Thread 4 Count: 81795. Execution time: 671 msec
```

```
Thread 5 Count: 80304. Execution time: 781 msec
Thread 6 Count: 79445. Execution time: 812 msec
Thread 7 Count: 78589. Execution time: 859 msec
Thread 8 Count: 77729. Execution time: 828 msec
Thread 9 Count: 77362. Execution time: 906 msec
Thread 10 Count: 76761. Execution time: 1000 msec
Thread 11 Count: 76174. Execution time: 984 msec
Thread 12 Count: 75828. Execution time: 1046 msec
Thread 13 Count: 75448. Execution time: 1078 msec
Thread 14 Count: 75235. Execution time: 1062 msec
Thread 15 Count: 74745. Execution time: 1062 msec
Thread 16 Count: 74475. Execution time: 1109 msec
Total primes: 1270606. Elapsed: 1188 msec
```

```
C:\Dev\Win10SysProg\x64\Release>PrimesCounter.exe 3 20000000 20
```

```
Thread 1 created (3 to 1000001). TID=30496
Thread 2 created (1000002 to 2000000). TID=7300
Thread 3 created (2000001 to 2999999). TID=50580
Thread 4 created (3000000 to 3999998). TID=21536
Thread 5 created (3999999 to 4999997). TID=24664
Thread 6 created (4999998 to 5999996). TID=34464
Thread 7 created (5999997 to 6999995). TID=51124
Thread 8 created (6999996 to 7999994). TID=29972
Thread 9 created (7999995 to 8999993). TID=50092
Thread 10 created (8999994 to 9999992). TID=49396
Thread 11 created (9999993 to 10999991). TID=18264
Thread 12 created (10999992 to 11999990). TID=33496
Thread 13 created (11999991 to 12999989). TID=16924
Thread 14 created (12999990 to 13999988). TID=44692
Thread 15 created (13999989 to 14999987). TID=53132
Thread 16 created (14999988 to 15999986). TID=53692
Thread 17 created (15999987 to 16999985). TID=5848
Thread 18 created (16999986 to 17999984). TID=12760
Thread 19 created (17999985 to 18999983). TID=13180
Thread 20 created (18999984 to 20000000). TID=49980
Thread 1 Count: 78497. Execution time: 218 msec
Thread 2 Count: 70435. Execution time: 343 msec
Thread 3 Count: 67883. Execution time: 421 msec
Thread 4 Count: 66330. Execution time: 484 msec
```

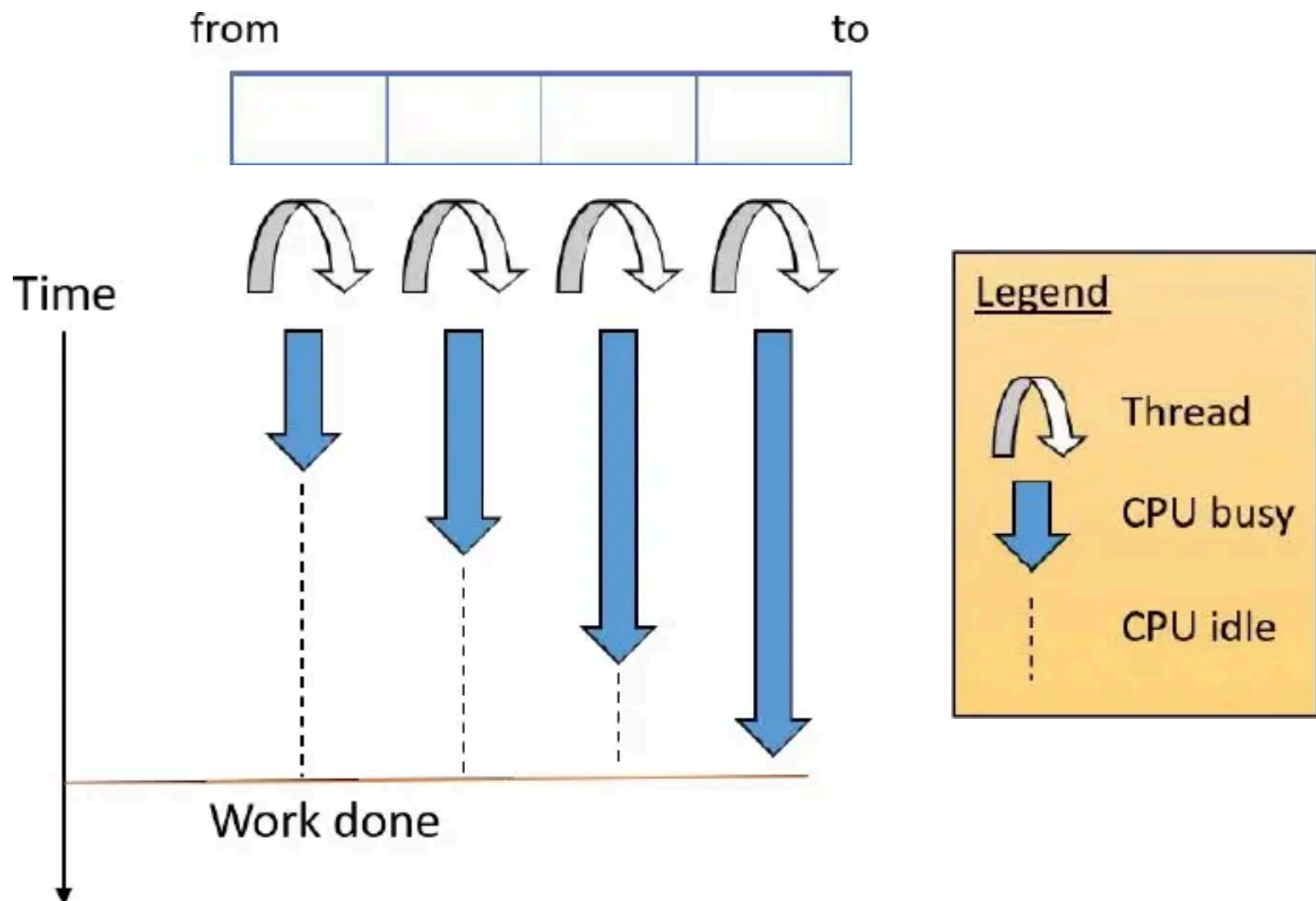
```
Thread 5 Count: 65366. Execution time: 578 msec
Thread 6 Count: 64337. Execution time: 640 msec
Thread 7 Count: 63798. Execution time: 640 msec
Thread 8 Count: 63130. Execution time: 703 msec
Thread 9 Count: 62712. Execution time: 718 msec
Thread 10 Count: 62090. Execution time: 703 msec
Thread 11 Count: 61937. Execution time: 781 msec
Thread 12 Count: 61544. Execution time: 812 msec
Thread 13 Count: 61191. Execution time: 796 msec
Thread 14 Count: 60826. Execution time: 843 msec
Thread 15 Count: 60627. Execution time: 875 msec
Thread 16 Count: 60425. Execution time: 875 msec
Thread 17 Count: 60184. Execution time: 875 msec
Thread 18 Count: 60053. Execution time: 890 msec
Thread 19 Count: 59681. Execution time: 875 msec
Thread 20 Count: 59560. Execution time: 906 msec
Total primes: 1270606. Elapsed: 1109 msec
```

The system on which these runs executed has 16 logical processors. Here is a couple of interesting observations from the above output:

- Improvement of execution time with increasing number of threads is not linear (not even close).
- Using more threads than the number of logical processors reduces execution time.

Why do we get these results? What is the optimal number of threads in a fork-join style algorithm? It seems that answer should be “the number of logical processors”, since and more threads will cause context switches to occur, because not all threads can execute at the same time, while using less threads would definitely leave some processors not utilized.

The reality, however, is not that simple. The single reason we get the two observations we did is this: the work is not split equally between the threads (in terms of execution time). This is simply because of the algorithm used: the larger the number, the more work has to be done because the `sqrt` function is a monotonic one and its output is in direct proportion to its input. This is often the challenge in fork-join algorithms: fair split of work. Figure 5-4 demonstrates what happens in an example case with four threads.

Figure 5-4: *Primes Counter* with 4 threads

Notice in the output above how later threads have longer running time, simply because they have more work to do. Now it may be obvious why we get better running time with 20 threads even if there are only 16 logical processors on the system. Early threads that finish quickly make processors free, allowing those “extra” threads (after 16) to get a processor, thus pushing work onward. Is there a limit? Of course, at some point, the context switch overhead, coupled with possible page faults because of more memory allocations for thread stacks will start making things worse. Clearly, asking what is the optimum number of processors for this program is not an easy question. And it can get even worse: this program only does CPU bound operations; no I/O. If threads need to do I/O from time to time, the question becomes even more difficult.

Terminating Threads

Every good (or bad) thread must at some point come to an end. There are three ways for a thread to terminate:

1. The thread function returns (best option)
2. The thread calls `ExitThread` (best to avoid)
3. The thread is terminated with `TerminateThread` (typically a bad idea)

The best option is to simply return from the thread function. When a thread starts execution, the thread function is not really the first or only function executed by the thread. The thread in fact starts execution inside an *NTDLL.dll* function named `RtlUserThreadStart`, which conceptually calls the thread's actual function as provided to `CreateThread`. Once the thread's function returns, `RtlUserThreadStart` does some cleanup and calls `ExitThread`. Note that `ExitThread` can only be called by a thread to terminate itself as its prototype suggests:

```
void ExitThread(_In_ DWORD exitCode);
```

`ExitThread` from *Kernel32.dll* is actually a forwarder to `RtlExitUserThread` in *NtDll.Dll*.

The problem with calling `ExitThread` explicitly from the thread's function is at least the fact that C++ destructors won't be called, as `ExitThread` never returns. So it's always better to simply return from the thread's function to allow it to cleanup local C++ objects properly.

In any case, `ExitThread` also calls the `DllMain` function for all DLLs in the process with the `DLL_THREAD_DETACH` reason argument. This allows DLLs to perform per-thread operations. For example, DLLs can allocate some block of memory to manage something on a per-thread basis. In many cases this is combined with *Thread Local Storage* (TLS), discussed in chapter 10.

The third option of terminating a thread is with a call to `TerminateThread` that can be made from another thread (even belonging to another process). The only condition is the caller's ability to obtain a handle to the thread with a `THREAD_TERMINATE` access mask. Here is the definition of `TerminateThread`:

```
BOOL WINAPI TerminateThread(  
    _Inout_ HANDLE hThread,  
    _In_     DWORD  dwExitCode);
```

Terminating a thread with this call is almost always a bad idea. The problem lies in what the thread managed to do and what it has not yet done because of its termination. If the thread is terminated while doing actual work, there is no way to tell what instructions it executed and what other code it could not execute because of termination. Application state could be in an inconsistent state. As an extreme (but not unlikely) example, the thread may have acquired a critical section (see chapter 7), and did not get the chance of releasing it, causing a deadlock, since other threads waiting for the critical section will wait forever.

Another issue with `TerminateThread` is that it does not call DLLs `DllMain` function with `DLL_THREAD_DETACH`. This means DLLs cannot run some code that might free memory or perform other actions to reverse what was done when the thread was created.

These problems with `TerminateThread` mean that calling this function safely is a rare occurrence, and there should be a better way to handle whatever scenario that seems to require it. Still, if this is desirable, the caller must obtain a powerful-enough handle having the `THREAD_TERMINATE` access right. Thread handles returned from `CreateThread` and `CreateProcess` always have full permissions. For other cases, obtaining a handle for an arbitrary thread can be attempted with `OpenThread`:

```
HANDLE OpenThread(  
    _In_ DWORD dwDesiredAccess,  
    _In_ BOOL bInheritHandle,  
    _In_ DWORD dwThreadId);
```

The function looks similar to `OpenProcess`, discussed in chapter 3. If the requested access mask can be obtained, a non-NULL handle is returned to the caller. If `THREAD_TERMINATE` is asked for and received, a call to `TerminateThread` is bound to succeed.

A Thread's Stack

Local variables and return addresses from functions reside on a thread's stack. The size of a thread's stack can be specified with the second parameter to `CreateThread`, but there are actually two values that affect a thread's stack: a reserved memory size that serves as the maximum size of the stack, and an initial, committed memory size, that is ready for use. The terms *reserved* and *committed* will be discussed in depth in chapter 12, but here's the gist of it: Reserved memory just marks a contiguous address space range as used for some purpose so that new allocations in the process address space won't be made from that range. For a stack, this is essential, as a stack is always contiguous. Committed memory means memory actually allocated, and so can be used.

It's possible to allocate the maximum stack size immediately, committing the entire stack upfront, but that would be wasteful, as a thread might not need the entire range for its stack related work. The memory manager has an optimization up its sleeve: commit a smaller amount of memory and if the stack grows beyond that amount, trigger an expansion of the stack, up to the reserved limit. The triggering is done by a page with a special flag, `PAGE_GUARD` that causes an exception if touched. This exception is caught by the memory manager, which then commits an additional page, moving the `PAGE_GUARD` page one page down (remember that a stack grows to lower addresses). Figure 5-5 shows this arrangement.

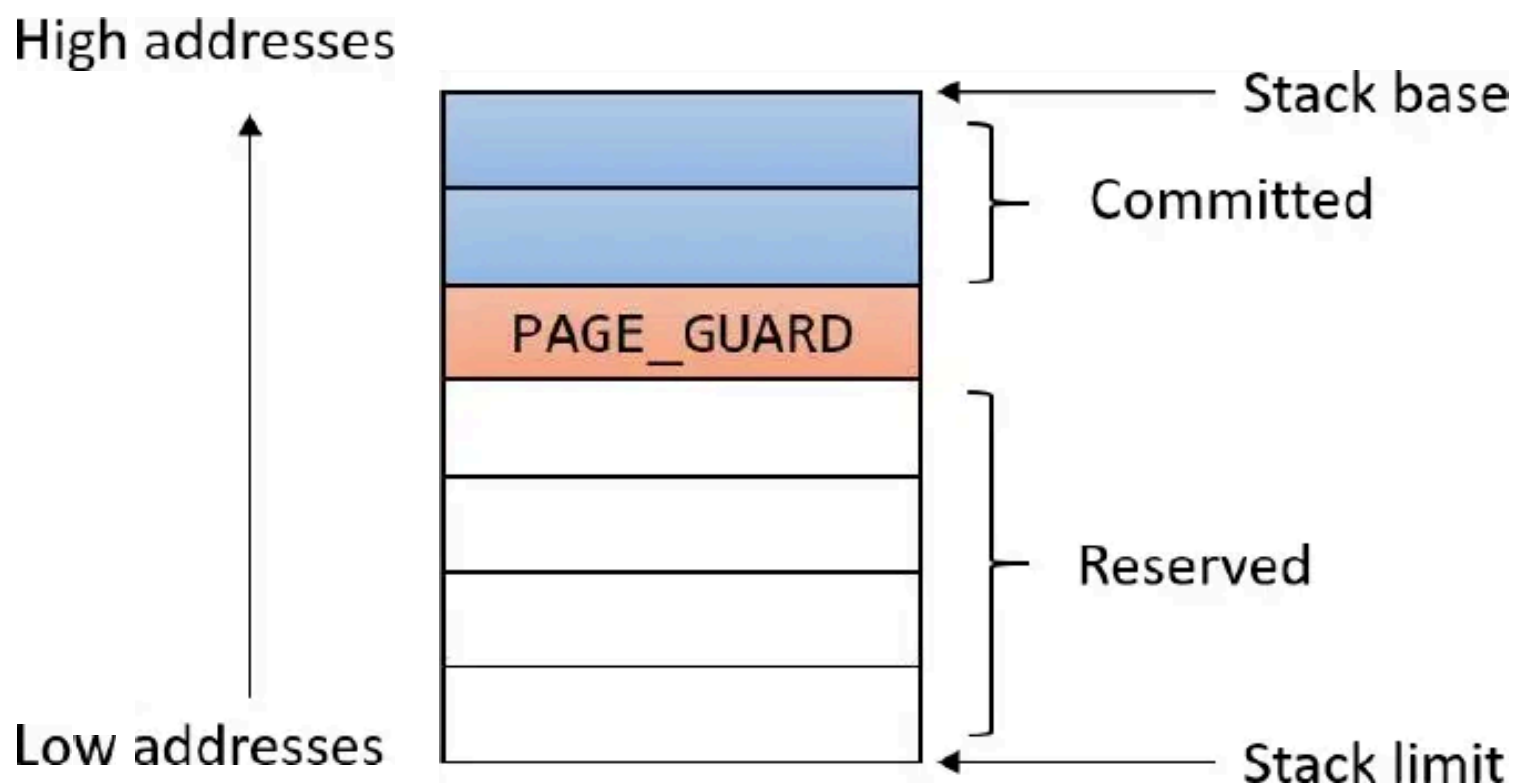


Figure 5-5: A thread's stack

The actual minimum for a guard page is 12 KB, meaning 3 pages. This guarantees that a stack expansion will allow at least 12 KB of committed memory to be available for the stack.

Typically, zero is passed as the stack (second) argument to `CreateThread`. In that case, the defaults for the committed and reserved sizes are retrieved from values stored in the Portable Executable (PE) header. The first thread, which is created by the kernel and so is out of our control always uses these values. You can dump these values using the *dumpbin* utility that is available as part of the Windows SDK. Here is an example with *Notepad*:

```
C:\>dumpbin /headers c:\windows\system32\notepad.exe
Microsoft (R) COFF/PE Dumper Version 14.24.28314.0
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Dump of file c:\windows\system32\notepad.exe
```

```
PE signature found
```

```
File Type: EXECUTABLE IMAGE
```

FILE HEADER VALUES

```
      8664 machine (x64)
      7 number of sections
9E7797DD time date stamp
      0 file pointer to symbol table
      0 number of symbols
F0 size of optional header
22 characteristics
      Executable
      Application can handle large (>2GB) addresses
```

OPTIONAL HEADER VALUES

```
...
      80000 size of stack reserve
      11000 size of stack commit
100000 size of heap reserve
      1000 size of heap commit
...
```

The default commit size for a thread's stack in *Notepad* is 0x11000 (68 KB) and the reserved size is 0x80000 (512 KB). These are the values used for *Notepad*'s first thread for sure. Other threads, explicitly created with `CreateThread` will have these values if the stack argument passed to `CreateThread` is zero.



You can also view this information in several free graphical tools, such as my own *PE Explorer v2*.

You can view this information with the *VMMap Sysinternals* tool. Run *Notepad*, and then

run *VMMap*. Select the *Notepad* process from the dialog (figure 5-6). Then click *OK*.

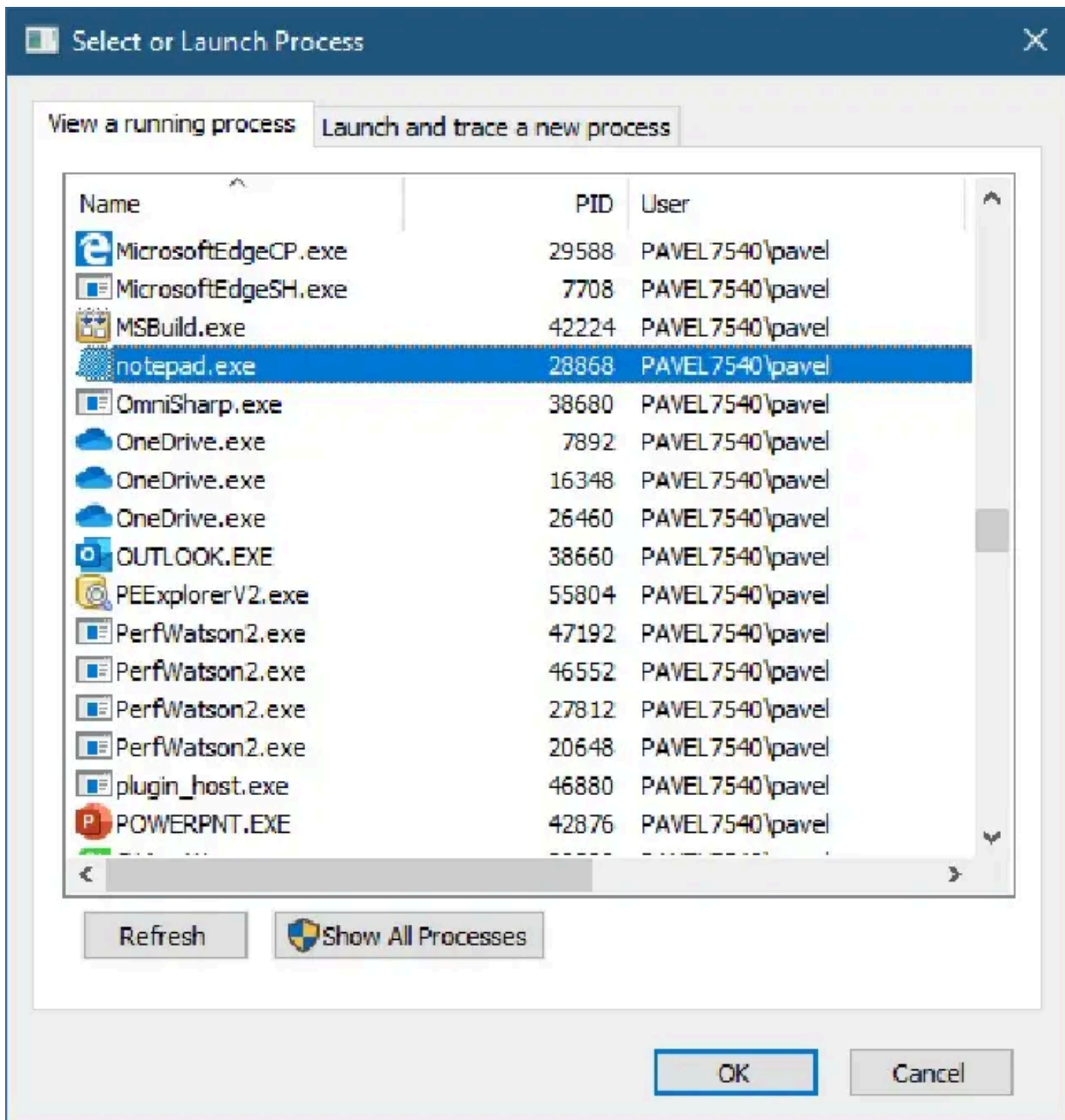


Figure 5-6: Process selector in *VMMap*

VMMap's main window opens. Select the *Stack* item in the middle list. This focuses the lower list to thread stacks only (figure 5-7).

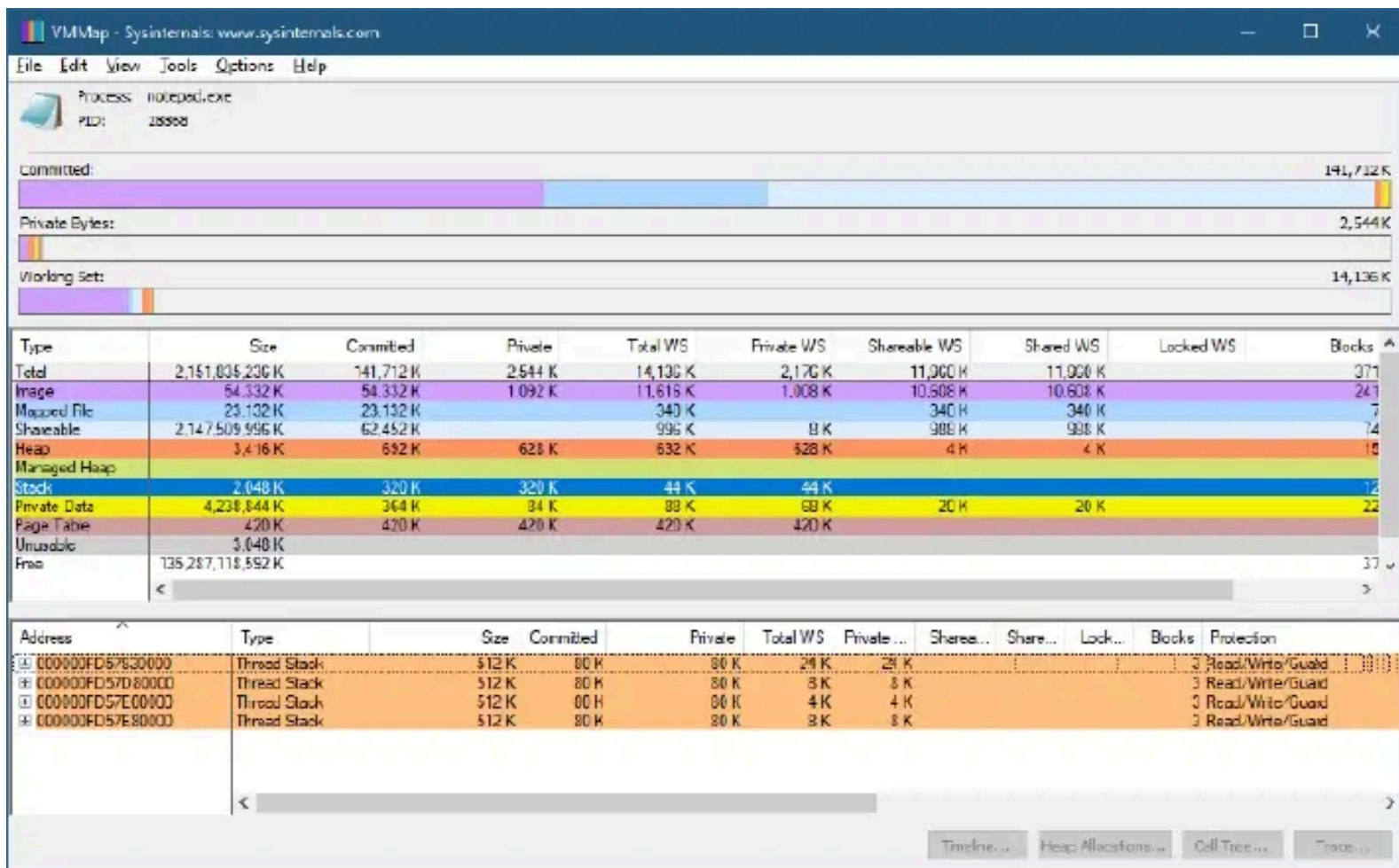


Figure 5-7: Selecting stacks in VMMap

Now open one of the stack items in the lower pane. You should see a committed size of 0x11000 bytes (68 KB) with a protection of Read/Write. Then a 12 KB guard page range, with the rest of the memory reserved (figure 5-8).

Address	Type	Size	Committed	Private	Total WS	Private ...	Share...	Share...	Lock...	Blocks	Protection
000000FD57930000	Thread Stack	512 K	80 K	80 K	24 K	24 K				3	Read/Write/Guard
000000FD57930000	Thread Stack	432 K									Reserved
000000FD579C0000	Thread Stack	12 K	12 K	12 K							Read/Write/Guard
000000FD579F0000	Thread Stack	68 K	68 K	68 K	24 K	24 K					Read/Write
000000FD57D80000	Thread Stack	512 K	80 K	80 K	8 K	8 K				3	Read/Write/Guard

Figure 5-8: One thread's stack in VMMap

VMMap will be discussed more thoroughly in chapter 12.

The `CreateThread` function has only one parameter for the stack size, and so it allows setting the initial committed memory or the maximum reserved memory, but not both. This is based on the `flags` argument. If it contains `STACK_SIZE_PARAM_IS_A_RESERVATION`, then the value is the reserved size; otherwise, it's the upfront committed size.

The fact that `CreateThread` only allows setting one of the values seems to be an oversight. The native function (from *NtDll*) `NtCreateThreadEx` allows setting both values.

Visual Studio allows changing the default stack sizes using the project's properties, under *Linker/System* node (figure 5-9). This simply sets the requested values in the PE header.

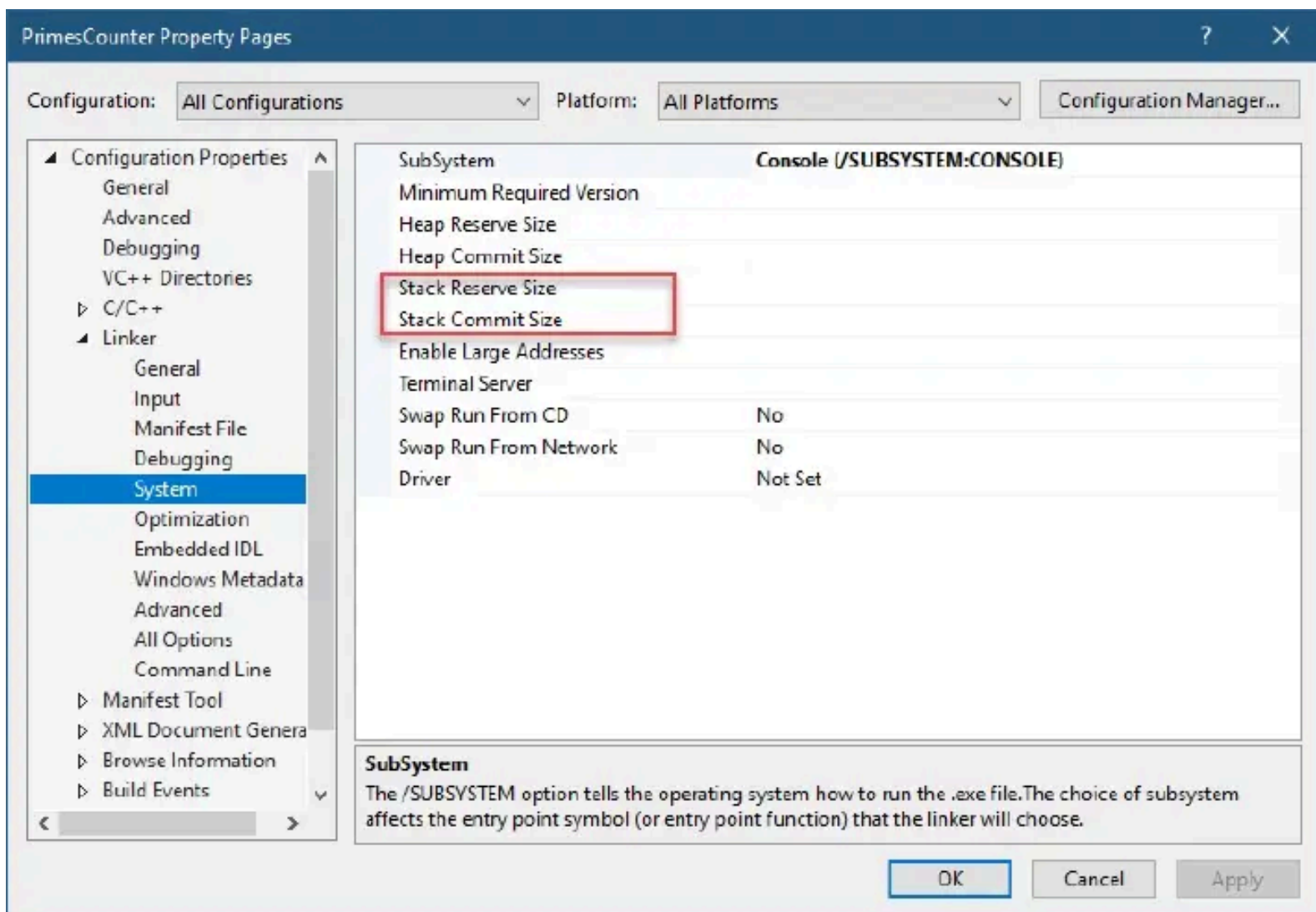


Figure 5-9: Stack sizes in Visual Studio

Finally, a thread can call `SetThreadStackGuarantee` to attempt at guaranteeing a certain stack size is available:

```
BOOL SetThreadStackGuarantee(_Inout_ PULONG StackSizeInBytes);
```

If the function succeeds, the increase in stack size is done by allocating more guard pages (which are also marked as committed), meaning they are guaranteed to be available if a stack expansion is required.

A Thread's Name

Starting with Windows 10 and Server 2016, a thread can have a string-based name or description, set with `SetThreadDescription`:

```
HRESULT SetThreadDescription(
    _In_ HANDLE hThread,
    _In_ PCWSTR lpThreadDescription);
```

The thread handle must have the `THREAD_SET_LIMITED_INFORMATION` access mask, which is easy to get for almost any thread. The name/description can be anything. Notice this function returns an `HRESULT`, where `S_OK` (0) means success. It's important to realize this is not the same as naming other kernel objects; there is no way to lookup a thread by its name/description. The name is just stored in the thread's kernel object and can be used as a debugging aid. Here is a simple example of setting the current thread's name:

```
::SetThreadDescription(::GetCurrentThread(), L"My Super Thread");
```

Visual Studio 2019 and later versions show the thread's name (if any) in the debugger's Threads window (figure 5-10).

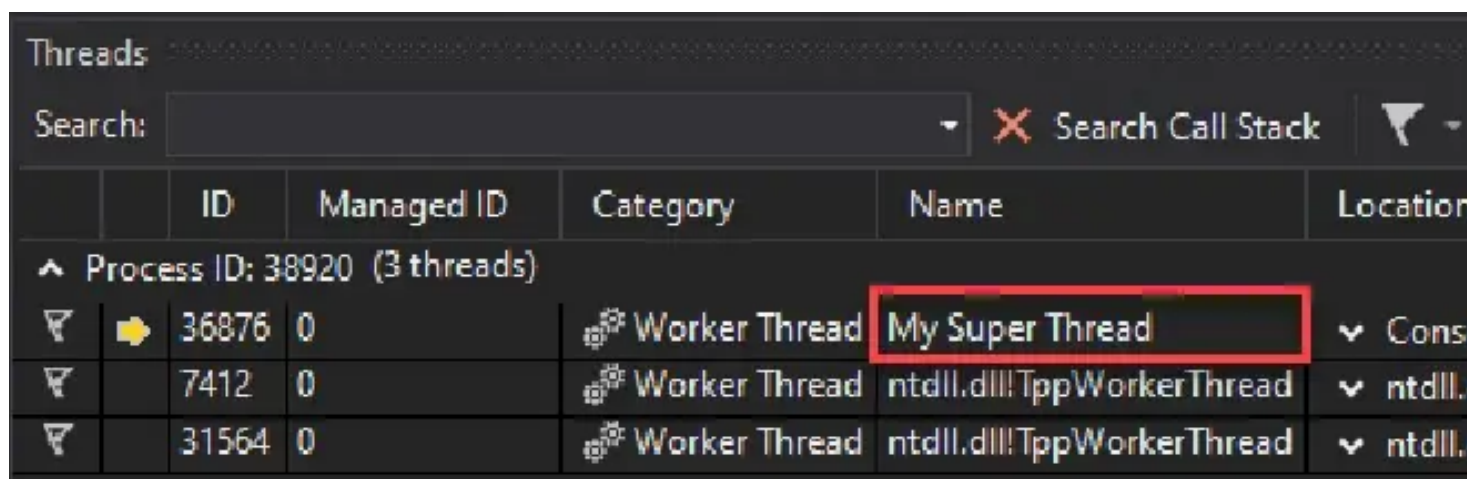


Figure 5-10: Thread's name in the Visual Studio debugger

Naturally, the opposite function exists as well:

```
HRESULT GetThreadDescription(  
    _In_ HANDLE hThread,  
    _Out_ PWSTR* ppszThreadDescription);
```

GetThreadDescription returns the result in a caller allocated pointer. The function requires calling LocalFree to free the memory it allocated. Here is an example:

```
PWSTR name;  
if (SUCCEEDED(::GetThreadDescription(::GetCurrentThread(), &name))) {  
    printf("Name: %ws\n", name);  
    ::LocalFree(name);  
}
```

What About the C++ Standard Library?

This book is about Windows programming, so directly discussing C++ may be out of place. Still, starting from the C++ 11 standard, the C++ standard library offers threading mechanisms (in fact, in earlier C++ standards, the word *thread* isn't even mentioned). The basic type is `std::thread` that allows creating a thread. Other classes deal with thread synchronization (see chapter 7); and there are more.

The greatest benefit of using the C++ standard library is the fact that it's, well, standard, which means it's cross-platform. If this is more important than other considerations, then by all means, go ahead and use it. The downside of the C++ standard library compared to working with the Windows API is the very few customizations available. The C++ standard library does not support thread priorities, affinities, CPU sets (all discussed in chapter 6), stack size control, and so on. This level of control can only be achieved by using Windows-specific APIs.

Exercises

1. Create a WTL dialog-based application that allows counting prime numbers (add edit boxes for number input) in a range of numbers. Perform the calculation on a separate thread so the UI thread is not blocked.
2. Add a *Cancel* button to the dialog that allows cancellation of the prime numbers counting mid-flight.

3. Create a Console application to calculate the Mandelbrot set with multiple threads concurrently, so the calculation is quicker. (You can find more on the Mandelbrot set in *wikipedia*.) The number of threads should be an input to the application, as well as the dimensions of the output bitmap. Divide the total number of lines by the number of threads, and assign each thread to the range of lines. Each pixel should be 0 (belongs to the set) or 1 (does not belong to the set). Store the results in a two-dimensional array.
4. Extend the application to write the output to a BMP or PPM format (both relatively simple), so the results can be viewed in a paint-like application.
5. Create a WTL application and calculate the Mandelbrot set with multiple threads without freezing the UI. Add the ability to pan/zoom and recalculate as needed.

Summary

In this chapter, we looked at the basics of thread creation and management. In the next chapter, we'll discuss thread scheduling and its associated properties, such as priority and affinity.